2024A7PS0002G
AI Lab3: Search

Section 5:

1) Formulation of Search problem:

   State S: The robot's current position in the warehouse
   Actions A: Move Up, Down, Left, or Right.
   Transition T: Moving the robot from its current cell to the adjacent cell in the chosen direction. (after making sure the destination cell is free)
   Initial state s0: S on the grid.
   Goal G: G on the grid.
   Cost c: Each move has a cost of 1.

   The robot's current row and column coordinates form the state. The problem is deterministic and a solution is defined by a series of valid steps that reach the goal from the initial state at the minimum cost.

2) Design of the agent:

   - A state will be represented by coordinates of (row, column).
   - The warehouse will be represented as a grid where # represents obstacles and [.] represents an empty spot.
   - An action is valid if the destination cell is empty.
   - The goal is for the agent to attain the coordinates of G.
   - Heuristic and path costs will be stored.
   - The final path will be reconstructed going backwards from G to S.

   When the program terminates, it should report whether a solution was found, the solution path, the path length, and the number of states expanded.

3) Program in py file

4) Prompts used:
   (along with warehouse image from the pdf)

   I am implementing a goal-based search agent in Python for a warehouse navigation problem. The warehouse is represented as an ASCII grid. S is the start position, G is the goal, # represents obstacles, and . represents free cells. The robot can move Up, Down, Left, or Right, with each movement having a cost of 1. Implement A* search using Manhattan distance as the heuristic: h(n) = |x - xG| + |y - yG| The program should: - represent grid positions as states; - avoid obstacles and invalid moves; - maintain a suitable frontier; - calculate g(n), h(n), and f(n); - avoid repeatedly expanding the same

state; - reconstruct the final path; - report whether a solution was found; - print the path and path length; - print the number of states expanded. Keep the code simple and explain the important parts.

Modify the warehouse navigation program to use Breadth-First Search instead of A*.

Use the same warehouse, movement rules, start position and goal.

Report:
- whether a solution was found;
- the path;
- the path length;
- the number of states expanded.

Keep the implementation simple so that its results can be compared directly with A*.

Modify the A* program so that I can test different heuristic functions.

Test:
1. h(n) = 0
2. Manhattan distance
3. Euclidean distance
4. 2 × Manhattan distance

For each version, report:
- whether a solution was found;
- the path length;
- the number of states expanded.

Do not change the warehouse map.

5) Results:

The agent passed all the specified cases:
Original warehouse, Trivial case (G next to S), No solution and Multiple solutions (with the goal of finding the best one.)

6) The BFS/A* comparison:

```
 BFS
--------------------------------------------------
Solution found: True
Path length: 40
States expanded: 64
Path:
[(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 5), (3, 5), (4, 5), (5, 5), (5, 6), (5, 7), (5, 8), (5, 9), (5, 10), (5, 11), (5, 12), (5,
13), (4, 13), (3, 13), (3, 12), (3, 11), (3, 10), (3, 9), (3, 8), (3, 7), (2, 7), (1, 7), (1, 8), (1, 9), (1, 10), (1, 11), (1, 12), (1,
 13), (1, 14), (1, 15), (2, 15), (3, 15), (4, 15), (5, 15), (6, 15), (7, 15)]

 A*
--------------------------------------------------
Solution found: True
Path length: 40
States expanded: 64
Path:
[(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 5), (3, 5), (4, 5), (5, 5), (5, 6), (5, 7), (5, 8), (5, 9), (5, 10), (5, 11), (5, 12), (5,
13), (4, 13), (3, 13), (3, 12), (3, 11), (3, 10), (3, 9), (3, 8), (3, 7), (2, 7), (1, 7), (1, 8), (1, 9), (1, 10), (1, 11), (1, 12), (1,
 13), (1, 14), (1, 15), (2, 15), (3, 15), (4, 15), (5, 15), (6, 15), (7, 15)]

Comparison
BFS path length: 40
A* path length: 40
BFS states expanded: 64
A* states expanded: 64
```

7) Heuristic Investigation:

Heuristic	Solution Found	Path Length	States Expanded
h(n) = 0	Yes	40	64
Manhattan distance	Yes	40	64
Euclidean distance	Yes	40	64
2 × Manhattan distance	Yes	40	64

8.1) It is important to formulate the search problem before writing the search algorithm, as the agent needs a clear design, which is built on it. An LLM can only generate appropriate code when the problem description provided is accurate and verbose.

8.2) A* is called an informed search algorithm because it uses additional information about the goal when deciding which state to explore next. It evaluates each state using the formula:

$$f(n) = g(n) + h(n)$$

where g(n) is the cost already travelled and
h(n) is an estimate of the remaining cost to the goal.

This heuristic information helps guide the search toward promising states rather than blind exploration.

8.3) The heuristic matters because it affects the order in which A* explores states.

8.4) The LLM helped me understand basic concepts I used to carry out the lab, like explaining A* clearly and detailing exactly how it works. I also used an LLM to generate code as per my specifications. It also helped me combine the code for all tasks into a simple program that can compare them as well.

8.5) If LLM-generated code were accepted without testing:

- It could contain bugs
- Incorrect assumptions due to lack of information provided in the prompt
- It might be overly complex
- It might fail edge cases without testing.